

WASM-4 / WAMR 运行时在 ESP32-C5 平台上的移植报告

版本 v1.1 · 2026-09-10 · 工程: led_01 (ESP32-C5 / ESP-IDF v5.5.5)

1. 概述

本报告记录将上游 MoonBit WASM-4 ESP32 示例工程（面向 ESP32-C6）移植到 ESP32-C5 平台的过程与结果。移植后，使用 MoonBit 编写的 WASM-4 游戏卡带（贪吃蛇，20,852 字节 WebAssembly 字节码）可在本平台上运行：由 WAMR 虚拟机解释执行卡带，由 WASM-4 运行时提供显示与输入抽象，画面输出至板载 2.0 英寸 ST7789 液晶屏（320×240）。

工作范围包括：芯片与构建目标适配、WAMR 运行时的构建接入、内存预算调整、显示驱动适配、控制台与人机接口接入、按键输入接入，以及与既有工程组件的系统集成。

目标芯片	游戏卡带	线性内存	固件大小
ESP32-C5（单核 RISC-V，240 MHz）	20,852 字节（wasm 字节码）	64 KB（1 个 wasm 页）	约 1.26 MB（app 分区 3 MB）

2. 系统架构

2.1 关系链路

- ① 游戏卡带 (.wasm)

MoonBit 编译产物 · 20,852 字节

导出 start / update / memory; 导入 blit / rect / trace / tone; 线性内存 64 KB

↓ wasm_runtime_load → instantiate → call_wasm
- ② WAMR 虚拟机

解释执行 · 导入解析

fast 解释器; 原生符号表 native_symbols[]

↓ 按函数名查表分派 (strcmp)
- ③ WASM-4 运行时

掌机规范实现

160×160 索引帧缓冲 (2bpp) ; 调色板、手柄字节

↓ 索引色 → RGB565 · 8 行分块
- ④ BSP 显示层

esp_lcd + 厂商 ST7789 驱动

SPI Mode 3 · 80 MHz; 窗口偏移 24、镜像与色序校正

↓ RGB565 像素流
- ⑤ 硬件

2.0" ST7789 液晶屏 · 320×240

该链路在游戏运行期间每帧执行一次（约 60 帧/秒）；卡带与宿主之间的契约由函数名建立。

输入路径：按键（GPIO7 / GPIO8，内部上拉，按下为低）→ 去抖与按下沿判定 → 宿主把“左转 / 右转”翻译为绝对方向键 → 写入卡带线性内存的手柄字节。

控制通道：USB-Serial-JTAG 命令行 → 显示自检、网络、存储、游戏状态查询，独立于游戏线程运行。

3. 移植内容

模块	上游（ESP32-C6 示例）	本工程（ESP32-C5）	移植动作
芯片与目标	ESP32-C6	ESP32-C5	构建目标切换；WAMR 平台层按 RISC-V 目标重新适配
Flash 与分区	8 MB，自定义分区表	4 MB， <code>partitions.csv</code> （app 3 MB）	按实际 Flash 重新规划分区
WAMR 接入	通过组件清单声明依赖（本地 vendor 路径）	作为独立组件，接入上游官方 ESP-IDF 集成	由官方 CMake/Kconfig 选择源文件、架构适配与 ABI，避免自定义编译宏与平台层不一致
显示	1.54" ST7789 240×240；上游自写 SPI 驱动，逐像素差分刷新	2.0" ST7789 320×240； <code>esp_lcd</code> 框架 + 厂商修复版驱动	重写显示层：驱动替换、面板参数校正、8 行分块流式贴屏
控制台	无（仅日志输出）	USB-Serial-JTAG 命令行（10 条命令）	新增 REPL 与命令表；主控制台由 UART 切换至 USB-Serial-JTAG
输入	4 个 GPIO 按键（组件化按键库）	2 个 GPIO 按键（GPIO7 / GPIO8，内部上拉，按下为低）	接入按键并做去抖；因游戏对转向有方向门禁，宿主侧将“左转 / 右转”翻译为游戏可接受的绝对方向键
启动流程	初始化后创建 FreeRTOS 任务运行游戏	屏幕初始化 → 游戏启动（独立线程）→ 控制台启动	游戏开机自启，独立于控制台运行；增加幂等守卫

3.1 内存适配

本平台可用堆内存约 145 KB，最大连续空闲块 128 KB。卡带声明 64 KB 线性内存，实例化时还需栈空间与运行时自身开销，因此内存预算按下列方式收敛：

最大连续空闲块		131,072 B
卡带线性内存		65,536 B

卡带（固件数据段）		20,852 B
实例化栈		8,192 B
显示流式缓冲（双缓冲）		5,120 B

决策	理由
宿主堆大小设为 0	WAMR 会将非零宿主堆向上取整为完整 wasm 页；若配置 32 KB 堆，实例化将一次性申请 128 KB，超过最大连续空闲块。卡带未导出内存分配接口，无需宿主堆。
帧缓冲改为流式分块	整帧 RGB565 缓冲需 50 KB；改为 8 行一块、双缓冲交替，占用 5 KB，并按块推送至显示驱动。
线程栈按需分配	初始化线程 5 KB、游戏线程 8 KB，按实际调用深度设定。

3.2 显示适配

本平台使用 2.0 英寸 ST7789 面板（320×240），与上游 1.54 英寸（240×240）不同，面板时序与窗口参数需要单独校正。显示层由四部分构成：

SPI 总线（Mode 3 · 80 MHz）→ esp_lcd 面板框架 → 厂商修复版 ST7789 驱动（符号遮蔽替换）→ 面板参数（偏移 24 / 镜像 / 色序）

项目	取值 / 说明
引脚	MOSI=GPIO0, SCLK=GPIO1, DC=GPIO2, RST=GPIO3, CS=GPIO4, 背光=GPIO9
逻辑分辨率	296×240（物理 320×240，横向窗口偏移 24 像素）
游戏画面	160×160 居中显示，偏移 (68, 40)
驱动来源	采用厂商提供的修复版 ST7789 驱动，以符号遮蔽方式替换框架默认实现

3.3 控制台与人机接口

上游示例无命令行界面。本工程接入 esp_console，主控制台由 UART 切换至 USB-Serial-JTAG（板载 Type-C 直连芯片），可在串口终端直接输入命令：

命令	说明
on / off	LED 开关，并写入 NVS 记忆状态
hello / restart	问候 / 软复位
scan / join	WiFi 扫描 / 连接
fetch / time	HTTP 数据获取 / 世界时间
lcd	显示自检
game	报告游戏运行状态
pin <gpio> <0 1>	引脚探测：把指定 GPIO 配成输出并回读实际电平（LCD / 电源 / BOOT 等关键脚受保护）

命令采用表驱动方式注册，新增命令为在命令表中追加一行。

3.4 启动时序

1. 上电，打印复位原因
2. LED 初始化（BSP）
3. LCD 初始化（幂等）并清屏
4. 按键初始化（输入上拉，按下为低；须先于游戏线程）
5. 游戏启动：加载卡带 → 实例化 → 查找 start / update → 创建独立游戏线程
6. 控制台启动，进入命令循环（游戏线程持续运行）

4. 移植方法

4.1 工程结构

```
led_01/
├─ main/                启动编排：屏幕初始化 → 游戏启动 → 控制台
├─ components/
│   ├─ BSP/             硬件抽象：LED + LCD（含厂商修复版驱动）
│   ├─ cli/             命令行：REPL 与命令表
│   ├─ wifi_app/        WiFi：扫描 / 连接
│   ├─ http_app/        HTTP：数据获取
│   ├─ game_app/        WASM-4 运行时 + FFI 接线 + 游戏卡带
│   └─ wamr/            WAMR 虚拟机（上游官方 ESP-IDF 集成）
└─ partitions.csv · sdkconfig
```

4.2 卡带构建链路

```
MoonBit 源码 (snake.mbt)

↓ moon build --target wasm

卡带 (snake.wasm, 20,852 字节)

↓ xxd → C 字节数组

gamecard.c (编入固件数据段)

↓ ESP-IDF 编译

固件 (运行时由 WAMR 从内存加载)
```

本工程使用上游预编译卡带，未改动游戏逻辑；若需修改游戏，使用 MoonBit 工具链重新编译并重新生成数组即可。

4.3 关键设计决策

决策	依据
WAMR 采用官方 ESP-IDF 集成	源文件选择、架构适配、ABI 与特性开关由上游统一维护，保证编译宏与平台实现一致
显示层使用框架 + 厂商驱动	通用驱动无法覆盖该面板的窗口偏移与镜像参数，厂商修复版可直接替换且不侵入框架
游戏独立线程运行	游戏与命令行解耦：控制台断开或重启不影响游戏；控制台负责诊断与配置
卡带按需加载、常驻内存	单张卡带一次加载；后续可通过网络下载卡带实现“换卡不重烧”

4.4 验证方式

手段	观察点
启动日志	卡带加载、实例化、start / update 函数地址输出
屏幕画面	160×160 游戏画面居中显示，帧率约 60 fps
命令行	game 报告运行状态；其余命令验证显示、网络、存储等功能
复位原因	启动时打印复位原因，用于区分设备复位与控制台重连

5. 资源占用与结果

项目	数值
固件大小	约 1.32 MB（app 分区 3 MB，剩余约 58%）
卡带体积	20,852 字节（约占固件 1.6%）
运行时内存占用	线性内存 64 KB + 栈 8 KB + 显示缓冲 5 KB + 线程栈 13 KB
显示刷新	160×160 帧缓冲，8 行分块推送，约 60 帧/秒
输入	2 个按键（GPIO7 / GPIO8），去抖 3 帧、按下沿触发；游戏可实际操控

6. 后续可扩展方向

方向	说明
卡带热加载	通过 WiFi 下载卡带至内存后加载，实现“换卡不重烧”
其他卡带	同一运行时可直接运行其他符合 WASM-4 规范的 wasm 卡带